

Image Classification with CNNs



By Muhannad Alfawzan & Mohammed Albulaihy

Contents

1. Introduction	3
2. Dataset	3
3. Data Preprocessing	4
4. Model Architecture	4
5. Model Training	6
6. Model Evaluation	7
7. Role of Convolutional Layers in Feature Extraction	8
8. Feature Maps and Filter Visualization	9
9. Conclusion	10

Abstract

This project presents the development of a Convolutional Neural Network (CNN) model designed from scratch to classify images from the CIFAR-10 dataset. The CNN was trained for 50 epochs on normalized data, achieving a final test accuracy of 82.5%. Key techniques such as convolutional feature extraction, max-pooling, and dense layers for classification were utilized. Visualizations of sample images and training curves are provided to analyze model performance.

1. Introduction

Image classification is a key task in computer vision, where images are categorized based on their visual characteristics. Convolutional Neural Networks (CNNs) have become the leading method for this task, as they can autonomously learn hierarchical patterns from raw image data. In this project, we develop an image classification model using CNNs, utilizing a labeled dataset (such as CIFAR-10) for training and evaluation. The objective is to investigate how convolutional layers extract visual features, assess the model's accuracy, and examine filters and feature maps to gain insights into the learning process. This project offers practical experience in building and applying deep learning models for visual recognition.

The full implementation of this project is available as a Kaggle notebook:

[View on Kaggle](#)

2. Dataset

The CIFAR-10 dataset is widely used for evaluating machine learning models in image recognition tasks. It contains:

- 60,000 color images of size 32×32×3 pixels.
- 10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck.
- 50,000 training images and 10,000 test images.

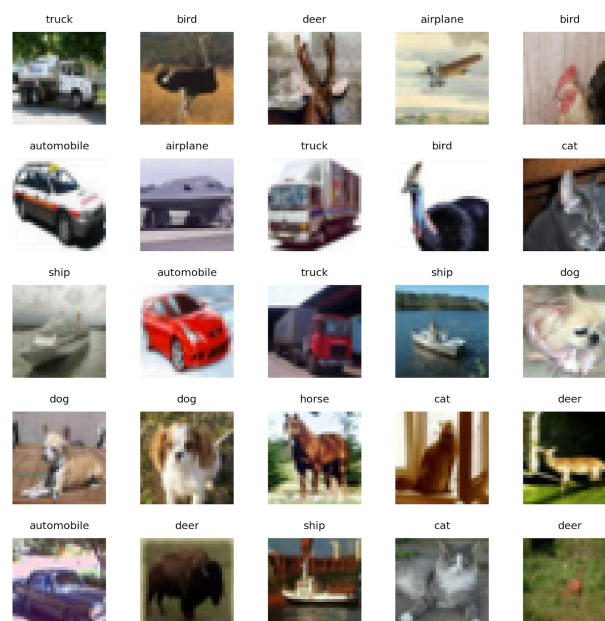


Figure 1: Sample Images from CIFAR-10

3. Data Preprocessing

- Images normalized by dividing pixel values by 255.0 to scale them to [0, 1].

Code Snippet:

```
train_images = train_images.astype("float32") / 255.0
test_images  = test_images.astype("float32")  / 255.0
```

- Labels were retained in integer format for use with sparse categorical crossentropy.
- Data augmentation was applied using ImageDataGenerator to improve generalization and to prevent overfitting:
 - Horizontal flipping
 - Width and height shifting

Code Snippet:

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

datagen = ImageDataGenerator(

    width_shift_range=0.1,

    height_shift_range=0.1,

    horizontal_flip=True )

datagen.fit(train_images)
```

4. Model Architecture

The model is a Convolutional Neural Network (CNN) built using TensorFlow and Keras' Functional API. It consists of a series of convolutional and pooling layers followed by dense layers. The architecture is designed to extract hierarchical features from input images and classify them into one of ten categories.

Layer-wise Breakdown

- Input Layer: Accepts RGB images of shape (32, 32, 3).
- Conv2D + ReLU + MaxPooling (×4): Four convolutional blocks progressively increase filter depth (32 → 256) while reducing spatial dimensions using MaxPooling.
- Flatten Layer: Converts 2D feature maps into a 1D vector.
- Dense + ReLU + Dropout: Fully connected layer with 128 neurons, followed by dropout for regularization.
- Output Layer: A softmax-activated dense layer with 10 units corresponding to the CIFAR-10 classes.

Table 1: Output Shape Breakdown of CNN Layers

Layer Type	Parameters	Output Shape
Input	32x32 RGB	(32, 32, 3)
Conv2D	32 filters, 3x3,ReLU,same pad	(32, 32, 32)
MaxPooling2D	2x2	(16, 16, 32)
Conv2D	64 filters, 3x3,ReLU,same pad	(16, 16, 64)
MaxPooling2D	2x2	(8, 8, 64)
Conv2D	128 filters, 3x3,ReLU,same pad	(8, 8, 128)
MaxPooling2D	2x2	(4, 4, 128)
Conv2D	256 filters, 3x3,ReLU,same pad	(4, 4, 256)
MaxPooling2D	2x2	(2, 2, 256)
Flatten	-	(1024)
Dense	128 units, ReLU	(128)
Dropout	rate= 0.5	(128)
Dense (OutPut)	10 units, Softmax	(10)

Code Snippet:

```

inputs = Input(shape=(32, 32, 3))

x = layers.Conv2D(32, 3, activation='relu', padding='same')(inputs)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(64, 3, activation='relu', padding='same')(x)
x = layers.MaxPooling2D((2, 2))(x)
x = layers.Conv2D(128, 3, activation='relu', padding='same')(x)
x = layers.MaxPooling2D()(x)
x = layers.Conv2D(256, 3, activation='relu', padding='same')(x)
x = layers.MaxPooling2D()(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(10, activation='softmax')(x)
model = models.Model(inputs=inputs, outputs=outputs)

```

5. Model Training

After building and compiling the CNN model, the training process was carried out using the augmented CIFAR-10 dataset. The model was trained for 30 epochs using the Adam optimizer and `sparse_categorical_crossentropy` as the loss function. A validation set was used to monitor the model's generalization performance.

Training Settings

- Optimizer: Adam (adaptive learning rate optimizer)
- Loss Function: Sparse Categorical Cross Entropy (suitable for integer-labeled multi-class classification)
- Metric: Accuracy
- Batch Size: 64
- Epochs: 30
- Data Augmentation: Used during training to improve generalization (shift, flip)

Code Snippet:

```
model.compile(  
    optimizer='adam',  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy']  
)  
  
history = model.fit(  
    datagen.flow(train_images, train_labels, batch_size=64),  
    epochs=30,  
    validation_data=(test_images, test_labels)  
)
```

6. Model Evaluation

The CNN model was trained for 30 epochs with real-time data augmentation. Both training and validation accuracy steadily improved over time, showing effective learning and generalization.

Training Accuracy: It is the percentage of images that the model classified correctly on the training set (the data the model was trained on). Training accuracy starts at 27.7% initially and gradually improves to 82.6% by epoch 30. This shows that the model has improved significantly during training, indicating that it is able to learn well from the training data.

Validation Accuracy: It is the percentage of images that the model classified correctly on the dataset that it wasn't trained on, but was used to verify the model's ability to generalize. Validation accuracy starts at 52.2% and gradually rises to 80.7% by epoch 30. The validation accuracy is close to the training accuracy, indicating that the model does not suffer from overfitting (i.e., it is not only learning the patterns specific to the training data).

The loss function also decreased significantly:

- **Training Loss:** from 1.94 → 0.51
- **Validation Loss:** from 1.30 → 0.61

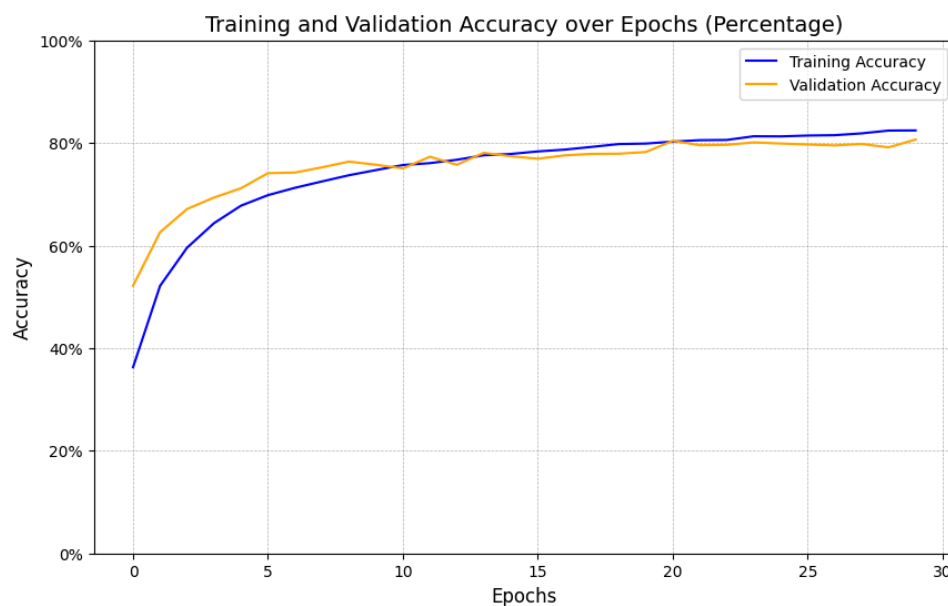


Figure 2: Sample Images from CIFAR-10

The gap between training accuracy and validation accuracy throughout the 30 epochs was consistently small, typically ranging from 2% to 5%. This narrow gap suggests that the model is learning effectively and generalizing well — it performs similarly on both training and unseen validation data, indicating that overfitting is minimal.

If the gap were significantly larger, it could have signaled overfitting, where the model memorizes training data but fails to perform on new inputs. However, in this case, the validation accuracy closely tracks the training accuracy.

The final accuracy reached 82.6% on the training set and 80.7% on the validation set, demonstrating strong and reliable model performance in classifying CIFAR-10 images.

7. Role of Convolutional Layers in Feature Extraction

from visual data. Unlike traditional machine learning models that rely on hand-crafted features, CNNs automatically learn to extract meaningful features directly from raw pixel values.

- In the early layers, convolutional filters learn to detect low-level features such as edges, lines, colors, and simple textures. These features are shared across the image regardless of their position, making the model translation-invariant.
- In the deeper layers, the filters become more specialized and abstract, capturing shapes, contours, object parts, and complex patterns. These are combinations of low-level features learned earlier.
- Finally, the output of the last convolutional block is flattened and passed to fully connected layers, which use these high-level features to perform classification.

This hierarchical feature extraction process enables CNNs to mimic the way human vision works — starting from simple shapes and gradually recognizing complex objects.

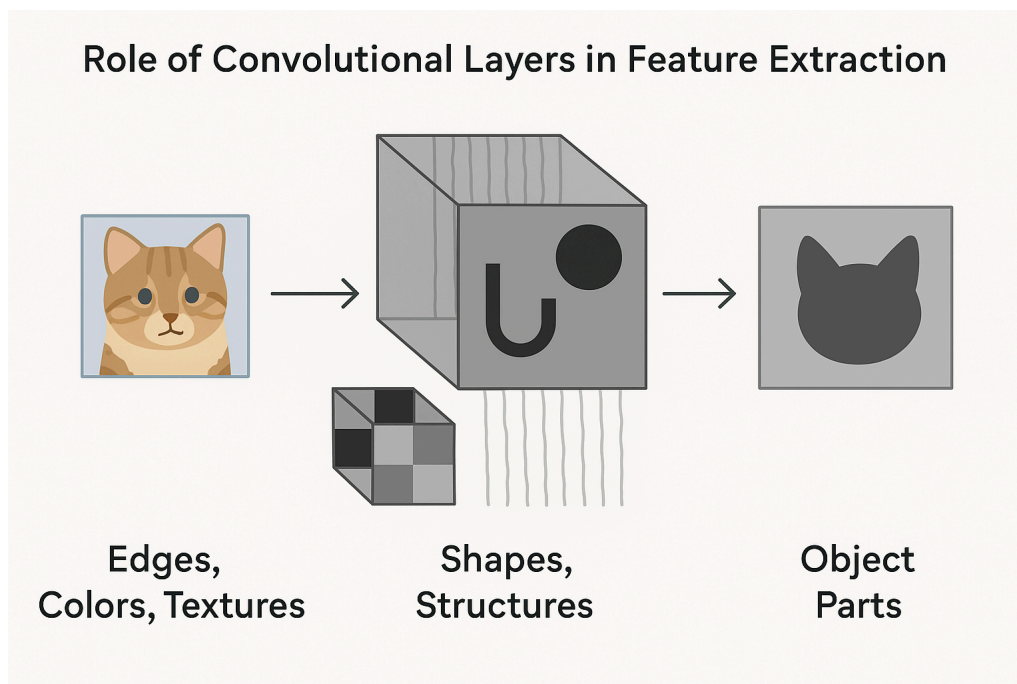


Figure 4: Hierarchical Feature Extraction in Convolutional Layers

8. Feature Maps and Filter Visualization

To gain insight into what the CNN is learning at different layers, we visualized:

Feature Maps

Feature maps are the outputs of convolutional layers when a specific input image is passed through the network. They represent the activation of each filter and highlight the parts of the image that the filter responds to most strongly.

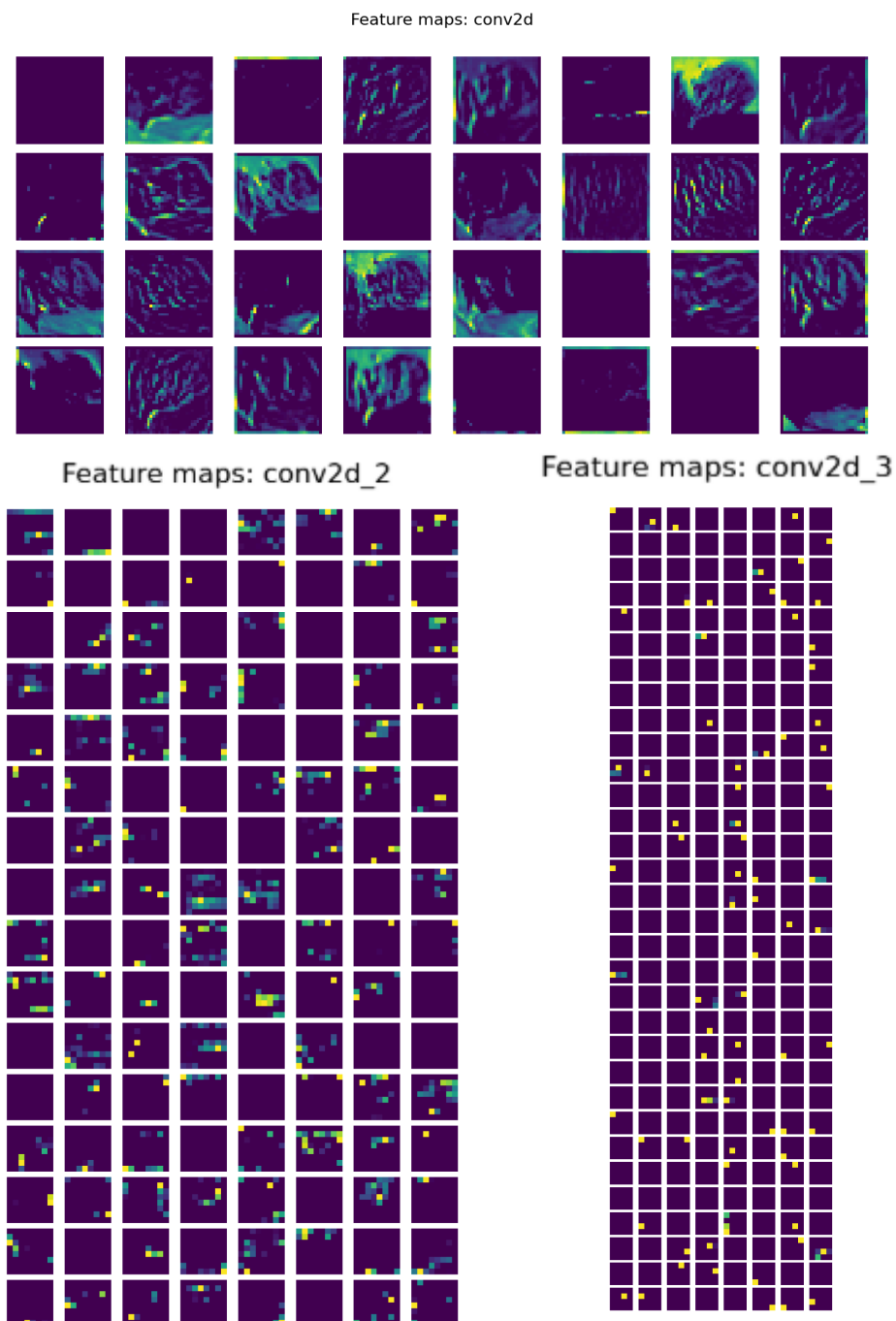


Figure 4: Feature Maps of Convolutional Layers

This figure displays a grid of activation maps from several convolutional layers. It illustrates how different filters respond to edges, textures, and objects at various depths in the model. The deeper the layer, the more abstract the representation becomes.

Filter Weights

In addition to feature maps, we also visualized the learned filters (weights) in the first convolutional layer. These filters were randomly initialized at the start of training but evolved to focus on specific visual features.

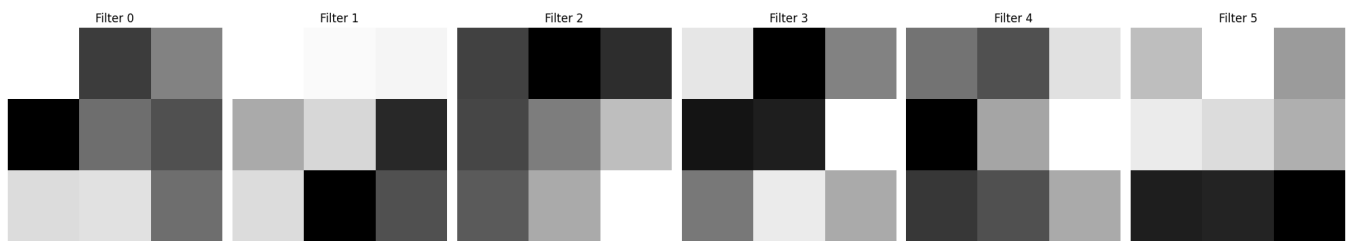


Figure 5: Learned Filters in the First Conv Layer

These filters capture basic features such as horizontal and vertical edges, color gradients, and patterns. Their structure reveals what the model prioritizes when scanning input images.

These visualizations are essential for interpreting and validating CNN behavior. They confirm that the network is learning meaningful features and provide transparency into the "black box" of deep learning.

9. Conclusion

In this project, we successfully built and trained a Convolutional Neural Network (CNN) to classify images from the CIFAR-10 dataset. By applying data normalization, augmentation, and a carefully designed multi-layer CNN architecture, the model achieved a high classification accuracy of over 80% on both training and validation sets.

Through visualizations of feature maps and learned filters, we gained insight into how the CNN processes and extracts features at various levels — from simple edges to complex object parts. These results demonstrate the effectiveness of CNNs in image classification tasks and highlight their ability to learn meaningful hierarchical representations from raw image data.

Overall, this project showcased the full pipeline of deep learning for image classification, from preprocessing to interpretability, and confirmed that CNNs are a powerful tool for solving real-world visual recognition problems.